

Full Stack Development with Angular and Spring Boot

Unit I:

Single-Page Application Architecture

Single-Page Applications (SPA)

Modern web applications must provide a fluid user experience — content should load or change dynamically based on even the smallest user interaction, without a full page reload, and the page must remain interactive with the user at all times. If a page takes too long to respond (e.g., 40+ seconds), users grow impatient and abandon the site for a competitor. This is why modern front-end frameworks are designed around the Single-Page Application model.

Definition

A Single-Page Application (SPA) is a web application that loads content dynamically based on user interaction, without refreshing the browser throughout the application's life cycle. The user is navigated through different logical views on the same page, saving the round trips a full page reload would require.

How Do We Achieve a Fluid Experience?

The Naive Approach	What Modern Web Apps Need
Load a brand-new HTML page for every interaction — a full browser refresh forces the creation of an entirely new DOM.	Update the existing DOM in place — only the parts of the page that changed are updated, keeping the page alive and responsive.
Result: slow, jarring transitions — the opposite of a fluid experience.	Frameworks like Angular (front end), paired with Spring Boot (back end), are built to support exactly this pattern.

What Is the DOM?

The Document Object Model (DOM) is a language-agnostic programming interface (API) for HTML and XML documents. It represents the document as a tree of objects that JavaScript (or any language that can talk to the browser) can read, manipulate, and style.

- The DOM is not the HTML file — it is the browser's live, in-memory representation, built only after the browser finishes parsing the HTML.

- Key idea: HTML tells the browser what to build; the DOM is what actually gets built, and it is the DOM — not the HTML file — that JavaScript works with.
- Angular's JavaScript interacts with the DOM to dynamically add, remove, or update elements/components, without reloading the page. This is precisely why Angular is called a Single-Page Application framework — the DOM is updated rather than a new HTML document being loaded for each navigation.

SPA Architecture — Traditional vs Single-Page Application

Request-Response Lifecycle

Step	Traditional Page Lifecycle	SPA Lifecycle
Initial load	Initial Request → ← full HTML page returned	Initial Request → ← full HTML page returned (once)
Subsequent interaction	Form POST → ← new HTML page	AJAX request → ← { JSON } data only
Result	Full page reload on every request	No reload — DOM updates in place

SPA Architecture / Working

1. The browser sends an initial request to the server and receives the complete HTML, CSS, and JS bundle (this loads all resources needed to navigate the entire site).
2. As the user interacts (clicks a link/button), instead of a full page request, an AJAX call is made to the server, which responds with only the required data (usually JSON).
3. Angular uses this data to update only the relevant part of the DOM in place — the rest of the page remains untouched.
4. The Angular Router updates the view and the address-bar URL to emulate navigation, but no new document request is actually made to the server.
5. This cycle repeats for every interaction, so the page never fully reloads.
6. 3. Important Points / Features (Traits of an SPA)
7. Loads all resources needed for the entire site on the first page load.
8. No page refresh throughout the application's life cycle.
9. The page remains interactive with the user at all times.
10. Dynamic view generation happens through DOM manipulation on the client; application data resides on the server and is fetched asynchronously (AJAX).
11. The Angular Router updates the URL to emulate navigation without a real server round-trip for HTML.

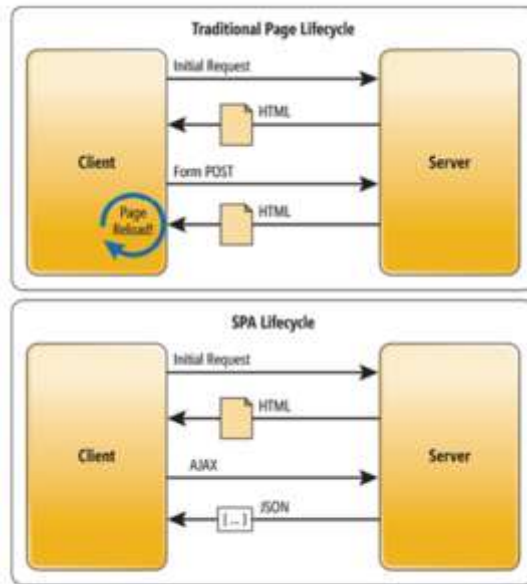


Figure 1.1: Traditional vs. SPA architecture Source: angular.io

Traditional Multi-Page App vs SPA — Comparison

Aspect	Traditional / Multi-Page Application	Single-Page Application (SPA)
Page structure	Multiple HTML pages (index.html, about.html, ...)	One HTML page; views are swapped dynamically
Resource loading	Loads only what the current page needs	Loads all resources needed to navigate the entire site on the first page load
Navigation	Full page reload on every navigation	No page refresh throughout the application life cycle
Server requests	A new document request is made on each click	Only the initial document request is made; later requests fetch data only (AJAX)
Visual experience	White flash during navigation	Smooth, uninterrupted navigation
Who updates the view	Browser requests a new page from the server	Angular Router updates the view on the client
Interactivity	Page is blocked/unresponsive while a new page loads	The page remains interactive with the user at all times

Introduction to TypeScript

TypeScript is a programming language developed by Microsoft that is a superset of JavaScript — every valid JavaScript program is also valid TypeScript. It adds static typing, classes, and interfaces on top of JavaScript, and compiles down to plain JavaScript that runs in any browser. Angular is a front-end development platform written entirely in TypeScript, and Angular applications are written in TypeScript as well.

Why Angular Chose TypeScript

- **No complex boilerplate:** TypeScript classes handle object-oriented structure cleanly — developers are not required to write complex code like manual constructor functions.
- **Errors caught early:** If TypeScript code contains an error, the developer is notified at an early stage — before the code ever runs in the browser.

Example — Early Error Detection

```
// sample.ts
let count: number = 0;
count = "oops"; // Error: Type 'string' is not assignable to type 'number'

$ node sample.ts
// Expected output: 0
```

TypeScript flags the type mismatch at compile time itself — before the program is ever run — preventing a bug from reaching production.

Benefits of TypeScript

- Familiarity for OOP developers — classes and interfaces make TypeScript comfortable for developers with an object-oriented background.
- **IntelliSense:** type checking and smart auto-completion as you type.
- **Basic Refactoring:** safely rename a symbol (variable, method, class) across an entire project.
- These language services work live in the editor, even without compiling.
- TypeScript's biggest selling point is tooling — advanced autocompletion, navigation, and refactoring — essential for large projects. Without strong language tooling, developers fear changing code, and large-scale refactoring becomes risky.

Example — TypeScript in an Angular Component

```
import { Component } from '@angular/core';

@Component({
```

```

selector: 'app-root',
templateUrl: './app.component.html'
})
export class AppComponent {
  title: string = 'hello-world-app';
}

```

- `@Component({...})` is a decorator — a TypeScript feature — that marks the class as an Angular component and links it to its template.
- `title: string = ...` declares a typed property; assigning a non-string value later would be flagged immediately by TypeScript.

Introduction to Angular

Angular is a front-end development platform written in TypeScript. It is a complete platform — not just a library — built entirely on TypeScript, and it is purpose-built for creating sophisticated Single-Page Applications, where the page updates dynamically without a full browser reload.

- SPA in one line: One HTML page. Angular swaps content in and out as you navigate. Only the relevant area (e.g., inside `<router-outlet>`) re-renders on navigation — no full page reload.
- Angular ships with well-integrated libraries covering routing, forms management, and client-server communication.

Building Your First Angular Application

Angular applications are created and managed using the Angular CLI (Command-Line Interface) — a tool used to initialize, develop, scaffold, and maintain Angular applications directly from a command shell.

Key Angular CLI Commands

Command	Purpose
<code>ng new <app-name></code>	Creates a new Angular project (project skeleton with source files, configuration, dependencies)
<code>ng serve</code>	Compiles the project, builds it in memory, starts the development server, and watches for changes
<code>ng generate component <name></code> (short: <code>ng g c <name></code>)	Generates a new component
<code>ng generate service <name></code> (short: <code>ng g s <name></code>)	Generates a new service
<code>ng build</code>	Compiles and bundles the app for production

Note: `ng serve` = Compile + Build (in memory) + Run Development Server.

Step-by-Step: Creating Your First App

- **Step 1 — Install prerequisites:** Ensure Node.js and NPM are installed, then install the Angular CLI globally: `npm install -g @angular/cli`. Verify with `ng version`.
- **Step 2 — Open the project folder in VS Code:** Open the integrated terminal (Terminal → New Terminal, shortcut `Ctrl + ``) — every Angular CLI command runs from here.
- **Step 3 — Create a new Angular application:** Run `ng new hello-world-app`. The CLI asks: Add Angular routing? → No (covered later); Which stylesheet format? → CSS.
- **Step 4 — Run the application:** `cd hello-world-app`, then `ng serve -o`. The `-o` flag builds the app and opens it automatically at `http://localhost:4200`.
- **Step 5 — Generate a component:** Run `ng generate component hello-world` (or `ng g c hello-world`). The CLI automatically creates the folder, files, and boilerplate code.

Files Generated for a Component

File	Usage
hello-world.component.ts	A TypeScript class with the <code>@Component</code> decorator (view-logic)
hello-world.component.html	The template (view)
hello-world.component.css	Styling for the component
hello-world.component.spec.ts	Test specification file

Note: A component is a self-contained unit — holding the view, styles, view-logic, and even its own test specification.

Plugging the New Component into the App

```
// app.component.ts (root component)

import { Component } from '@angular/core';
import { HelloWorldComponent } from './hello-world/hello-world.component';

@Component({
  selector: 'app-root',
  imports: [HelloWorldComponent],
  template: `
    <h2>Simple App Demo</h2>
    <app-hello-world></app-hello-world>
  `,
})
export class AppComponent { }
```

- AppComponent is the root component every Angular app starts from — it is bootstrapped automatically when the app runs.
- `<app-hello-world></app-hello-world>` is the selector of HelloWorldComponent. Angular replaces this tag with the component's rendered output, proving components are reusable, drop-in building blocks.

```
// hello-world.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-world.component.html',
  styleUrls: ['./hello-world.component.css']
})
export class HelloWorldComponent { }

<!-- hello-world.component.html -->
<h3>Hello World</h3>
<h4>This is Hello World Component</h4>
```

- ❖ HelloWorldComponent's selector: 'app-hello-world' is exactly what makes `<app-hello-world></app-hello-world>` work inside AppComponent's template — Angular replaces that tag with this component's rendered HTML.
- ❖ AppComponent imports HelloWorldComponent from './hello-world/hello-world.component' and lists it in its own imports: [HelloWorldComponent] array (standalone component syntax) — without this import, Angular would not recognize `<app-hello-world>` as a valid tag in AppComponent's template.

Angular Building Blocks

Angular applications are built by combining a set of core building blocks: Component Architecture, Angular Components, Data Binding, Directives, and Pipes. Together, these let an application be organised into small, reusable, self-contained pieces of functionality.

Component Architecture

Why Do We Need Components? — The Problem with Combining Features

Suppose a website needs two independent features: a **Calendar** (to check holidays) and a **Calculator** (to calculate loan EMI). If both features are built by combining their view and logic together into shared layers — instead of keeping them as separate, self-contained units — several hidden problems arise:

Problem	Explanation (in the context of the Calendar/Calculator example)
No Flexibility	The Calendar and Calculator are locked together in the same page/layer, so neither can be placed or used independently on a different page without dragging the other along.
No Reusability	If another application needs only the Calendar (e.g., a scheduling app), the Calendar cannot be pulled out and reused there, because its code is fused with the Calculator's.
Poor Maintainability	If a bug is fixed or logic is changed in the Calculator, there's a risk of breaking the Calendar too, since both share the same underlying code/layer.
Poor Extensibility	If a new feature needs to be added to just the Calendar (e.g., holiday reminders), it cannot be added independently — because the Calendar isn't a separate, self-contained unit.

The Solution: Separate, Self-Contained Units

Keeping the Calendar and Calculator as two **separate, self-contained components** — each wrapping its own functionality and dependencies — solves every problem above:

Quality	How Separation Solves It
Flexibility	Each component can be placed on any page independently — the Calendar can be used on one page and the Calculator on another.
Reusability	The Calendar component can be picked up and reused in a completely different application, without carrying the Calculator along with it.
Maintainability	A change or bug fix inside the Calculator's code has no effect on the Calendar — because they are entirely separate units.
Extensibility	New features can be added to the Calendar alone, without touching or risking the Calculator's code.

Definition: A component is a self-contained unit that wraps its functionality and dependencies together. An architecture built from such units is called **component architecture** — the foundation of every Angular application.

Relating This to Traditional MVC

- **Traditional MVC approach:** Model, View, and Controller are three separate layers. Building both features this way combines the Calendar's view and view-logic together with the Calculator's, in the same View/Model/Controller layers.
- **The catch:** Because the Calendar and Calculator logic/views are fused together in shared layers, neither can be pulled apart and reused elsewhere. If another application needs only the Calendar, traditional MVC blocks that reuse.
- **What we actually want:** Pick-and-drop reusability — plug the Calendar or Calculator into any page, anywhere. This is exactly what Angular's component architecture provides.

Angular Components

Components are the main building blocks in Angular. Every Angular component is made of two parts, wrapped together:

- **A TypeScript class:** holds the component's view logic — variables (state) and methods that define what the component does.
- **The @Component decorator:** attached right above the class; specifies the component's view — an HTML template plus styles.

Hands-On: Creating a Component

```
ng generate component hello-world
# or the short form:
ng g c hello-world

// hello-world.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello-world',
  templateUrl: './hello-world.component.html',
  styleUrls: ['./hello-world.component.css']
})
export class HelloWorldComponent { }

<!-- hello-world.component.html -->
<h3>Hello World</h3>
<h4>This is Hello World Component</h4>
```

The component's selector (app-hello-world) can then be used anywhere in the app, like any other HTML tag, and Angular will render this component wherever the tag appears — this is what makes components reusable, drop-in building blocks.

Understanding Data Binding

Data binding connects the component's TypeScript properties with its HTML template, keeping the view and the model automatically in sync.

Type of Binding	Syntax	Direction
Text Interpolation	{{ value }}	Component → View
Property Binding	[property] = "value"	Component → View
Event Binding	(event) = "handler"	View → Component
Two-Way Binding	[(ngModel)] = "property"	View ⇌ Component

Note: Every binding type (other than interpolation) uses the equal sign =, with the punctuation on the target's left telling Angular the direction of data flow.

Text Interpolation

The simplest form of binding — displays a component property's value directly in the template. Wrap the property name in double curly braces: {{ propertyName }}. Used for titles, labels, messages — any plain text content.

```
// component class
export class HelloWorldComponent {
  title = 'Hello World';
}

<!-- template -->
<h3>{{ title }}</h3>
```

Property Binding

Property binding moves a value in one direction — from a component's property into a target element's property. The target element property is enclosed in square brackets: [property] = "expression".

```
isDisabled: boolean = true;

<button [disabled]="isDisabled">Disabled Button</button>
```

Example: a button has a disabled property. When true, the button cannot be clicked. We create a boolean isDisabled in the component class and bind it to the button's disabled property.

Event Binding

Event binding lets a component respond when a user interacts with the page. Data flows from the DOM (view) to the model (component). It has three parts: the event source (any DOM element), the

target event (click, hover, change, etc.), and the event handler (a function that runs the code for that event).

```
export class ToggleEffectComponent {
  isOn: boolean = true;
  doToggle() {
    this.isOn = !this.isOn;
  }
}

<button (click)="doToggle()">{{ isOn ? 'ON' : 'Off' }}</button>
```

- Target event: click — syntax is (click) on the left of =.
- Template expression: doToggle() — the handler method that runs on click.
- Each click flips isOn between true and false, and the caption updates automatically through interpolation.

Two-Way Binding (brief introduction, will be covered later)

Two-way binding combines property binding and event binding in a single syntax, [(ngModel)] — informally called 'banana in a box'. Data flows in both directions: the view updates when the model changes, and the model updates when the user changes the view (e.g., typing in an input box).

Understanding Angular Directives

A directive is an Angular feature that adds behaviour to an HTML element or changes how that element is displayed or works — an instruction telling Angular to add special behaviour to an HTML element.

Types of Directives

Type	Description	Example
Component Directive	A directive with its own template — every Angular component is technically a directive.	<app-hello-world></app-hello-world>
Attribute Directive	Changes the appearance or behaviour of an element, without changing DOM structure.	ngStyle, ngClass
Structural Directive	Changes the structure of the DOM by creating, removing, or repeating elements/views based on a condition or collection.	*ngIf, *ngFor, *ngSwitch

1. Component Directive

A **component** is a special type of directive that has its own template.

```
// typescript
@Component({
  selector: 'app-hello',
  template: '<h2>Hello</h2>'
})
export class Hello {}
```

- @Component → decorator
- Hello → the component class; conceptually, a **specialized Angular directive**
- So @Component — at the framework/conceptual level is a component and is a specialized directive **with a template**. But at the code level, we don't write @Directive *and* @Component together — we simply write @Component(...), and Angular understands this is a component.

Purpose: Create and control a section of the UI.

Usage:

```
html
<app-hello-world></app-hello-world>
```

This *creates/renders its own UI*, because the component has a template attached.

2. Attribute Directive

Definition: An attribute directive is an Angular directive that is applied as an attribute to an **existing** HTML element to change its appearance or add behavior.

Built-in Attribute Directive — ngClass

```
// typescript
import { Component } from '@angular/core';
import { NgClass } from '@angular/common';
@Component({
  selector: 'app-root',
  imports: [NgClass],
  template: `
```

```

<h2>ngClass Example</h2>

<p [ngClass]="{'highlight': isHighlighted}">
  Hello Students!
</p>

<button (click)="isHighlighted = !isHighlighted">
  Change Color
</button>
`,
styles: `
.highlight {
  color: red;
  font-size: 25px;
}
`,
}))
export class App {
  isHighlighted = false; }

```

- `[ngClass]="{'highlight': isHighlighted}"` conditionally applies the CSS class `highlight` to the `<p>` element whenever `isHighlighted` is true.
- Clicking **Change Color** toggles `isHighlighted`, which adds/removes the `highlight` class — turning the text red and larger, since Angular re-evaluates the binding automatically.
- **Output:** Initially "Hello Students!" is plain; after clicking "Change Color," the text turns red and bigger.

Built-in Attribute Directive — `ngStyle`

```

//typescript
import { Component } from '@angular/core';
import { NgStyle } from '@angular/common';
@Component({

```

```

selector: 'app-root',
imports: [NgStyle],
template: `
  <h2>ngStyle Example</h2>
  <p [ngStyle]="{
    'color': isHighlighted ? 'red' : 'black',
    'font-size': isHighlighted ? '30px' : '20px'
  }">
    Hello Students!
  </p>

  <button (click)="isHighlighted = !isHighlighted">
    Change Style
  </button>
  `
})
export class App {
  isHighlighted = false; }

```

- `[ngStyle]="{...}"` binds an object of CSS style properties directly to the element — here color and font-size change based on the `isHighlighted` condition, using inline ternary expressions.
- Unlike `ngClass` (which toggles a predefined CSS class), `ngStyle` lets you set **individual style properties directly**, computed dynamically in the template.

3. Structural Directives

Definition: A structural directive is an Angular directive that changes the structure of the DOM by creating, removing, or repeating elements or views based on a condition or collection.

a) *ngIf

```
//typescript
@Component({
  selector: 'app-root',
  imports: [NgIf],
  template: `
    <h2>Hello World</h2>

    <button (click)="isVisible = !isVisible">
      Show / Hide
    </button>

    <p *ngIf="isVisible">
      Hello Students!
    </p>
  `,
})
export class App {
  isVisible = true;
}
```

- *ngIf="isVisible" creates the <p> element in the DOM only when isVisible is true; when false, the element is **completely removed** from the DOM (not just hidden).
- Clicking **Show/Hide** toggles isVisible, which adds or removes the <p>Hello Students!</p> element from the page in real time.

b) *ngFor

```
//typescript
import { Component } from '@angular/core';
import { NgFor } from '@angular/common';

@Component({
  selector: 'app-root',
  imports: [NgFor],
  template: `
    <h2>Student List</h2>

    <ul>
      <li *ngFor="let student of students">
        {{ student }}
      </li>
    </ul>
  `
})
export class App {
  students = ['Anu', 'Ravi', 'Priya', 'Kiran']; }
```

- *ngFor="let student of students" repeats the element once for **every item** in the students array — dynamically generating one per student:

students

↓

*ngFor

↓

Anu → creates Anu

Ravi → creates Ravi

Priya → creates Priya

Kiran → creates Kiran

- **Output:** A bulleted list showing Anu, Ravi, Priya, Kiran.

c) *ngSwitch

```
//typescript
@Component({
  selector: 'app-root',
  imports: [NgSwitch, NgSwitchCase, NgSwitchDefault],
  template: `
    <h2>Student Grade</h2>

    <div [ngSwitch]="grade">
      <p *ngSwitchCase="'A'">Excellent</p>
      <p *ngSwitchCase="'B'">Very Good</p>
      <p *ngSwitchCase="'C'">Good</p>
      <p *ngSwitchDefault>Invalid Grade</p>
    </div>
  `
})
export class App {
  grade = 'B';
}
```

- [ngSwitch]="grade" evaluates the grade property once, and Angular checks it against each *ngSwitchCase value.
- Only the matching <p> is rendered in the DOM — here, since grade = 'B', only <p>Very Good</p> appears.
- *ngSwitchDefault renders as a fallback only if none of the *ngSwitchCase values match.
- **Output:** "Very Good" is displayed on the page.

Summary Table

Directive	Type	Effect
@Component	Component directive	Creates and controls a section of the UI (own template)
[ngClass]	Attribute directive	Conditionally applies/removes a CSS class
[ngStyle]	Attribute directive	Conditionally applies individual inline style properties
*ngIf	Structural directive	Adds/removes an element from the DOM based on a condition
*ngFor	Structural directive	Repeats an element once per item in a collection
*ngSwitch	Structural directive	Renders only the matching case's element from a set of options

Pipes

A pipe is a simple function used inside a template to transform data into a desired, display-friendly format, without changing the actual data stored in the component. Pipes are applied using the pipe operator `|`.

```
{{ expression | pipeName }}  
{{ expression | pipeName:'argument' }}
```

Commonly Used Built-in Pipes

Pipe	Purpose
DatePipe	Formats a date value (shortDate, mediumDate, longDate, fullDate, or a custom pattern like 'dd/MM/yyyy')
UpperCasePipe / LowerCasePipe / TitleCasePipe	Converts text case
SlicePipe	Extracts a portion (substring/subarray) of the data
DecimalPipe / CurrencyPipe / PercentPipe	Formats numbers as decimals, currency, or percentages
JsonPipe	Converts an object into its JSON string representation — useful for debugging

Date Pipe — Example

```
import { Component } from '@angular/core';  
import { DatePipe } from '@angular/common';
```

```
@Component({
  selector: 'app-root',
  imports: [DatePipe],
  template: `
    <p>Default: {{ today | date }}</p>
    <p>Short Date: {{ today | date:'shortDate' }}</p>
    <p>Custom: {{ today | date:'dd/MM/yyyy' }}</p>
  `,
})
export class App {
  today = new Date();
}
```

- `today | date` applies the `DatePipe` with its default format.
- `today | date:'shortDate'` passes `'shortDate'` as an argument, producing a compact format like `8/20/26`.
- The `today` variable in the class itself is never modified — only its displayed representation changes.

Custom Pipes

Angular also allows creating custom pipes for application-specific formatting needs, scaffolded using the CLI:

```
ng generate pipe greet
```

This creates a pipe class file that can be imported and used in any component's template, just like a built-in pipe.

More on Pipes in Angular

Definition:

- Pipes are used to **transform and format data** in an Angular template before displaying it.
- Pipes use the **pipe symbol** `( | )`.
- They make data easier to **read and present** to the user.

Syntax:

html

```
{{ value | pipeName }}
```

Angular provides several built-in pipes, such as: uppercase, lowercase, titlecase, date, currency, number, percent, json, slice.

1. Date Pipe

Formats and Example Outputs:

Format	Example Output
shortDate	8/20/26
mediumDate	Aug 20, 2026
longDate	August 20, 2026
fullDate	Thursday, August 20, 2026
short	8/20/26, 12:00 AM
medium	Aug 20, 2026, 12:00:00 AM
long	August 20, 2026 at 12:00:00 AM GMT+5:30
full	Thursday, August 20, 2026 at 12:00:00 AM GMT+05:30

Example:

```
// typescript
import { Component } from '@angular/core';
import { DatePipe } from '@angular/common';

@Component({
  selector: 'app-root',
  imports: [DatePipe],
  template: `
    <h2>Date Pipe Example</h2>
    <p>Default: {{ today | date }}</p>
    <p>Short Date: {{ today | date:'shortDate' }}</p>
    <p>Medium Date: {{ today | date:'mediumDate' }}</p>
    <p>Long Date: {{ today | date:'longDate' }}</p>
```

```

<p>Full Date: {{ today | date:'fullDate' }}</p>
<p>Custom: {{ today | date:'dd/MM/yyyy' }}</p>
<p>Custom: {{ today | date:'dd-MM-yyyy' }}</p>
<p>Custom: {{ today | date:'dd MMMM yyyy' }}</p>
,
})
export class App {
  today = new Date();
}

```

Output:

Default: Aug 20, 2026

Short Date: 8/20/26

Medium Date: Aug 20, 2026

Long Date: August 20, 2026

Full Date: Thursday, August 20, 2026

Custom: 20/08/2026

Custom: 20-08-2026

Custom: 20 August 2026

- today | date applies the default date format.
- Passing an argument after : (e.g., 'shortDate') changes the output format.
- A custom pattern string (e.g., 'dd/MM/yyyy') gives full control over how the date is displayed.

2. JSON Pipe

Example:

```

//typescript
import { Component } from '@angular/core';
import { JsonPipe } from '@angular/common';

```

```

@Component({
  selector: 'app-root',
  imports: [JsonPipe],
  template: `
    <h2>JSON Pipe Example</h2>
    <p>Without JSON pipe:</p>
    <p>{{ student }}</p>
    <p>With JSON pipe:</p>
    <p>{{ student | json }}</p>
    <p>Only Name:</p>
    <p>{{ student.name }}</p>
  `,
})
export class App {
  student = {
    name: 'Anu',
    age: 20
  };
}

```

Output:

Without JSON pipe: [object Object]

With JSON pipe: {"name":"Anu","age":20}

Only Name: Anu

- Without the json pipe, Angular cannot display an object directly in the template — it prints [object Object].
- The json pipe converts the object into its **JSON string representation**, making it readable — useful for debugging.

- Individual properties (like student.name) can still be accessed and displayed directly without the pipe.

3. Decimal, Currency and Percent Pipe

Example:

```
//typescript
import { Component } from '@angular/core';
import { DecimalPipe, CurrencyPipe, PercentPipe } from '@angular/common';
@Component({
  selector: 'app-root',
  imports: [DecimalPipe, CurrencyPipe, PercentPipe],
  template: `
    <h2>Number, Currency and Percent Pipes</h2>

    <p>Number: {{ number | number }}</p>
    <p>Currency: {{ price | currency:'INR' }}</p>
    <p>Percentage: {{ percentage | percent }}</p>
  `
})
export class App {
  number = 1234567.89;
  price = 2500;
  percentage = 0.75;
}
```

Output:

Number: 1,234,567.89

Currency: ₹2,500.00

Percentage: 75%

- number pipe formats a plain number with grouping separators (commas) and decimal places.
- currency:'INR' formats the value as Indian Rupee currency, adding the ₹ symbol and two decimal places.
- percent converts a decimal fraction (e.g., 0.75) into a percentage display (75%).

4. UpperCase, LowerCase, TitleCase, and Slice Pipe

Example:

```
//typescript
import { Component } from '@angular/core';
import {
  UpperCasePipe,
  LowerCasePipe,
  TitleCasePipe,
  SlicePipe
} from '@angular/common';
@Component({
  selector: 'app-root',
  imports: [
    UpperCasePipe,
    LowerCasePipe,
    TitleCasePipe,
    SlicePipe
  ],
  template: `
    <h2>Angular Pipes Example</h2>
    <p>Original: {{ name }}</p>
    <p>Uppercase: {{ name | uppercase }}</p>
```



```

<p>Lowercase: {{ name | lowercase }}</p>

<p>Titlecase: {{ name | titlecase }}</p>

<p>Slice: {{ name | slice:0:5 }}</p>
,
})

export class App {
  name = 'hello students';
}

```

Output:

Original: hello students

Uppercase: HELLO STUDENTS

Lowercase: hello students

Titlecase: Hello Students

Slice: hello

- uppercase / lowercase convert the entire string to upper or lower case.
- titlecase capitalises the first letter of every word.
- slice:0:5 extracts a portion of the string — here, characters from index 0 up to (but not including) index 5, giving "hello".

Summary Table

Pipe	Purpose	Example
date	Formats a date value	{{ today date:'shortDate' }}
json	Converts an object to its JSON string form	{{ student json }}
number	Formats a number with grouping/decimals	{{ number number }}

Pipe	Purpose	Example
currency	Formats a number as currency	{{ price currency:'INR' }}
percent	Formats a decimal as a percentage	{{ percentage percent }}
uppercase lowercase	Converts text case	{{ name uppercase }}
titlecase	Capitalises first letter of each word	{{ name titlecase }}
slice	Extracts a portion of a string/array	{{ name slice:0:5 }}

Angular Component Lifecycle Hooks

Introduction

Every Angular component moves through a **managed lifetime**, not just a single render. Angular exposes key moments in this lifetime so developers can **hook into them** — running custom code at exactly the right point.

What Angular does with a component:

#	Phase	Description
1	Create	Instantiates the component class
2	Render	Renders the component's view and any child component views
3	Update	Updates views and data-bound instances when properties change
4	Remove	Removes the component and its view from the DOM
5	Destroy	Destroys the component instance and frees resources

Why Lifecycle Hooks Matter

The Interface Pattern

- Angular provides **one lifecycle hook interface per phase**.
- Each interface contains a **single hook method**, named by prefixing the phase with **ng**.

- **Example:** to tap into initialization, implement the `OnInit` interface and its method `ngOnInit()`.
- Angular instantiates the component via its **constructor**, then invokes each hook you've implemented at the right point in the instance's life.

With Lifecycle Hooks You Can:

1. Initialize component instances
2. Trigger change detection when needed
3. Respond to updates during change detection
4. Clean up before an instance is destroyed

The Sequence Angular Guarantees — Birth to Teardown

1. `ngOnChanges()`
2. `ngOnInit()`
3. `ngDoCheck()`
4. `ngAfterContentInit()`
5. `ngAfterContentChecked()`
6. `ngAfterViewInit()`
7. `ngAfterViewChecked()`
8. `ngOnDestroy()`

1. `ngOnChanges()` — Responds to Input Changes

What It Does: Invoked whenever Angular sets or resets a data-bound input property. The method receives a `SimpleChanges` object holding the **current and previous values** of each changed property — so you can react precisely to what changed.

Called: Before `ngOnInit()` (if bound inputs exist), and on every subsequent input change.

Key Detail: Runs before `ngOnInit()` on first load if the component has bound inputs, and again any time one or more `@Input()` properties change afterward.

Example:

typescript

```
// parent.component.ts
export class ParentComponent {
  count = 10;
  increase() {
    this.count++;
  }
}
```

html

```
<!-- parent.component.html -->
<p>Parent Count: {{ count }}</p>
<button (click)="increase()">Increase</button>
<app-child [myCount]="count"></app-child>
```

// typescript

```
// child.component.ts
import { Component, Input, OnChanges, SimpleChanges } from '@angular/core';
export class ChildComponent implements OnChanges {
  @Input() myCount!: number;
  ngOnChanges(changes: SimpleChanges) {
    console.log("myCount changed");
    console.log(changes);
  }
}
```

- Every time the parent's count changes, Angular re-evaluates `[myCount]="count"` and calls `ngOnChanges()` in the child, passing a `SimpleChanges` object describing exactly what changed.

2. `ngOnInit()` — Initializes the Component

What It Does: Used for initializing the component. Angular calls it after first displaying the data-bound properties and setting the component's input properties — the standard place to put setup logic that depends on those inputs.

Called: Once, after the first `ngOnChanges()`.

Key Detail: Fires exactly once per component instance, and still runs **even if `ngOnChanges()` never gets called** because the component has no data-bound input properties.

Example:

```
// typescript
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `<h2>Hello Angular</h2>`
})
export class App implements OnInit {
  ngOnInit() {
    console.log('App component initialized');
  }
}
```

3. `ngDoCheck()` — Custom Change Detection

What It Does: Provided to detect and act upon changes that Angular's own change detection cannot or would not catch on its own — for example, mutations inside an object or array that don't create a new reference.

Called: Immediately after `ngOnChanges()` on every run, and after `ngOnInit()` on the first run.

Key Detail: Runs on **every single change-detection cycle**, so keep the logic inside lightweight — it executes far more often than the other hooks.

4. ngAfterContentInit() — Responds to Projected Content

What It Does: Responds after Angular projects external content — content passed in via <ng-content>, from a parent template — into the component's view. This is the right moment to work with that projected content.

Called: Once, after the first ngDoCheck().

Key Detail: Only relevant to components that use **content projection**; fires a single time, right after the content is first checked.

Example (Content Projection Setup):

```
//html
<!-- Usage -->
<app-child>
  <p>Hello</p>
</app-child>
html
<!-- child.component.html -->
<div>
  <h2>Child Component</h2>
  <ng-content></ng-content>
</div>
```

- The <p>Hello</p> passed between <app-child> tags is **projected** into the <ng-content> slot — ngAfterContentInit() fires right after this projected content is first initialized.

5. ngAfterContentChecked() — Rechecks Projected Content

What It Does: Provided to respond after Angular checks the content that was projected into the component. It runs alongside every change-detection pass that follows the initial content check.

Called: After ngAfterContentInit(), and after every subsequent ngDoCheck().

Key Detail: Pairs with ngAfterContentInit() — one for the **first** check, this one for **every check thereafter**.

6. `ngAfterViewInit()` — Responds to Rendered Views

What It Does: Responds after Angular initializes the component's own view and any child views. This is the **earliest safe point** to access `@ViewChild` references, since the view is now fully rendered.

Called: Once, after the first `ngAfterContentChecked()`.

Key Detail: Fires exactly once, after **both** the component's view **and** all of its children's views exist.

7. `ngAfterViewChecked()` — Rechecks Rendered Views

What It Does: Provided to respond after Angular checks the component's view and its children's views. It runs on every change-detection cycle that follows the initial view check.

Called: After `ngAfterViewInit()`, and after every subsequent `ngAfterContentChecked()`.

Key Detail: Pairs with `ngAfterViewInit()` the same way `ngAfterContentChecked()` pairs with `ngAfterContentInit()` — first check vs. every check after.

8. `ngOnDestroy()` — Cleans Up Before Removal

What It Does: Provided so you can write cleanup code just before the component instance is destroyed. Use it to unsubscribe from observables, detach event handlers, clear timers, and release any other resources.

Called: Immediately before Angular destroys the component.

Key Detail: Skipping cleanup here is one of the **most common sources of memory leaks** in Angular applications — always pair a subscription or listener with teardown logic here.

Example:

```
// typescript
// child.component.ts
import { Component, OnDestroy } from '@angular/core';
@Component({
  selector: 'app-child',
  template: `
    <h2>Child Component</h2>
    <p>I am the child component.</p>
  `
})
```

```

})

export class ChildComponent implements OnDestroy {

  ngOnDestroy() {

    console.log('Child component is destroyed');

  }

}

```

```

// typescript
// app.component.ts (parent)
export class AppComponent {

  showChild = true;

}

html

<!-- app.component.html -->

<h2>Parent Component</h2>

<button (click)="showChild = !showChild">Show / Hide Child</button>

@if (showChild) {

  <app-child></app-child>

}

```

- A component is **destroyed** when Angular removes it from the application/view.
- Clicking "Show / Hide Child" toggles showChild; when it becomes false, Angular removes <app-child> from the DOM, triggering ngOnDestroy() in the child and logging 'Child component is destroyed'.

Grouping the Eight Hooks by Purpose

Creation	Content Projection	View Rendering	Destruction
ngOnChanges()	ngDoCheck()	ngAfterViewInit()	ngOnDestroy()
ngOnInit()	ngAfterContentInit()	ngAfterViewChecked()	
	ngAfterContentChecked()		

The Big Idea: Hooks vs Change Detection

Hooks don't all detect changes. **Different hooks correspond to different stages of Angular's component lifecycle.**

Angular calls lifecycle hooks *during* a change-detection cycle.

Hooks are not the mechanism that performs change detection.

- **Change detection** = Angular's checking mechanism.
- **Any hook** = a lifecycle event that lets **you** run custom checks during that process.
- Angular's change detection cycle is **triggered automatically** whenever Angular has reason to check the application.

Quick Revision Summary

Hook	Fires	Purpose
ngOnChanges()	Before ngOnInit(), then on every input change	React to @Input() changes
ngOnInit()	Once, after first ngOnChanges()	Component initialization
ngDoCheck()	Every change-detection cycle	Custom change detection
ngAfterContentInit()	Once, after first ngDoCheck()	React to projected content (first time)
ngAfterContentChecked()	After every subsequent ngDoCheck()	React to projected content (every check)
ngAfterViewInit()	Once, after first ngAfterContentChecked()	React to rendered views (first time) — safe for @ViewChild

Hook	Fires	Purpose
ngAfterViewChecked()	After every subsequent ngAfterContentChecked()	React to rendered views (every check)
ngOnDestroy()	Immediately before destruction	Cleanup — unsubscribe, clear timers, detach listeners

Unit II: Services and Dependency Injection

Angular Services

A service is any value, function, or feature that an application needs, which carries out a particular, well-defined task of the application. Services handle tasks that do not involve the view — they belong to the application layer, as opposed to components, which belong to the view layer.

Aspect	Components	Services
Layer	View layer	Application layer
Responsibility	Presenting application data in the view	Non-view tasks: fetching data, validation, logging, authentication
In short	Handle WHAT the user sees	Handle everything ELSE the app needs to do

Note: Components delegate non-view tasks to services instead of doing that work themselves — this dependency is fulfilled through Dependency Injection.

Dependency Injection (DI)

Dependency Injection (DI) is a design pattern in which an object's dependencies are supplied to it from outside, rather than the object creating those dependencies itself. In Angular, a component usually depends on one or more services — these services are the component's dependencies — and Angular's DI system supplies (injects) the required service instances into components automatically.

The Idea Behind DI

Consider a simple analogy: a user needs to find the square root of a number, so the user needs a calculator. We say the user object has a dependency on a calculator object — the calculator has a method `findSqrRoot(n1)`, and the user must first get hold of the calculator before calling it.

Two Ways to Get a Dependency

Approach	Code	Who Creates the Object
Without DI	<pre>let c = new Calculator(); c.findSqrRoot(userNumber);</pre>	The user (class) creates the object itself

With DI	// c is supplied by the framework c.findSqrRoot(userNumber);	Angular creates and supplies (injects) the object
---------	---	---

Note: The key idea of DI: the user is not creating the object of the dependency (calculator) — instead, the dependency is injected into the user object by the framework itself.

How DI Helps Build Maintainable Applications

Benefit	Explanation
Flexibility	An implementation can be swapped without touching the code that uses it
Modularity	Components and services stay cleanly separated
Reusability	The same service can be shared across many components
Testability	Dependencies can be mocked easily for unit tests

Example: Calculator Service via DI

Step 1 — Create the service:

```
// calculator.service.ts
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class CalculatorService {
  findSqrRoot(num: number) {
    return Math.sqrt(num);
  }
}
```

Step 2 — Inject the service into a component (constructor is used to inject the service or dependency):

```
// simple-ui.component.ts
import { Component } from '@angular/core';
import { CalculatorService } from './calculator.service';

@Component({
  selector: 'app-simple-ui',
  template: `
    <input #num type="number">
    <button (click)="calculate(num.value)">Find Square Root</button>
    <p>Result: {{ result }}</p>
  `,
})
```

```
export class SimpleUiComponent {
  result: number = 0;

  constructor(private calculator: CalculatorService) {}

  calculate(value: string) {
    this.result = this.calculator.findSqrRoot(Number(value));
  }
}
```

Explanation

- `@Injectable({ providedIn: 'root' })` registers `CalculatorService` with Angular's DI system at the root level, making it available application-wide.
- `constructor(private calculator: CalculatorService) {}` — instead of writing `new CalculatorService()`, the component simply declares its dependency here; Angular's DI system automatically creates and supplies the instance.
- `this.calculator.findSqrRoot(...)` — the component asks the injected service to perform the calculation; the component's own job is only to receive user input and display the result.

Here's the same `SimpleUiComponent`, rewritten to use `inject()` instead of constructor injection:

```
// simple-ui.component.ts
import { Component, inject } from '@angular/core';
import { CalculatorService } from '../calculator.service';

@Component({
  selector: 'app-simple-ui',
  template: `
    <input #num type="number">
    <button (click)="calculate(num.value)">Find Square Root</button>
    <p>Result: {{ result }}</p>
  `,
})
export class SimpleUiComponent {
  result: number = 0;
```

```
// Inject service without constructor

private calculator = inject(CalculatorService);

calculate(value: string) {
  this.result = this.calculator.findSqrRoot(Number(value));
}
}
```

What changed:

- `import { Component } from '@angular/core';` → now also imports `inject`.
- `constructor(private calculator: CalculatorService) {}` is **removed entirely**.
- `private calculator = inject(CalculatorService);` directly requests the `CalculatorService` instance from Angular's DI system at the point of declaration.
- `calculate(value: string)` stays exactly the same — it still calls `this.calculator.findSqrRoot(...)`, since `calculator` refers to the same injected service instance either way.

Both versions behave identically at runtime — same shared `CalculatorService` instance (since it's registered with `providedIn: 'root'`), same output. Only the syntax used to *obtain* the dependency differs.

Data flow: User enters number → Component receives it → Component asks `CalculatorService` → Service calculates (`Math.sqrt`) → Component displays the result.

Quick Revision Summary

- SPA = one HTML page + dynamic DOM updates via AJAX/JSON, no full reloads.
- TypeScript = superset of JavaScript; adds static typing, classes, tooling; Angular is written in it.
- Component = TypeScript class + `@Component` decorator (view-logic + view).
- Data binding = interpolation `{{ }}`, property `[ ]`, event `( )`, two-way `[ ( ) ]`.
- Directives = component / attribute / structural (`*ngIf`, `*ngFor`).
- Pipes = transform display data using `|`, without changing the underlying value.
- Lifecycle hooks = `ngOnChanges` → `ngOnInit` → `ngDoCheck` → `ngAfterContentInit` → `ngAfterContentChecked` → `ngAfterViewInit` → `ngAfterViewChecked` → `ngOnDestroy`.

- View encapsulation = Emulated (default) / ShadowDom / None — controls CSS scoping.
- Change detection = Angular's automatic mechanism to sync view with model.
- Services = non-view application tasks (fetch, validate, log, authenticate).
- Dependency Injection = framework supplies service instances to components automatically, via the constructor.

Few Questions

1. Explain the architecture of a Single-Page Application (SPA). How does it differ from a traditional multi-page web application?
2. Explain TypeScript and discuss its importance in Angular application development.
3. Explain the role of components, directives, and pipes in an Angular application.
4. Explain Angular component architecture with a suitable example. Discuss how data binding is used between the component and its template.
5. Explain the steps involved in building your first Angular application.
6. What are Angular directives? Explain the different types of directives with suitable examples.
7. Explain the following with respect to Angular components: Lifecycle hooks
8. Explain Dependency Injection (DI) in Angular. Describe how dependency injection helps in developing maintainable applications.
9. Explain dependency injection through the constructor and the inject() property.